

ПОЛНАЯ ИНСТРУКЦИЯ: SIGNALS & SLOTS (пошагово)

ЧТО ТАКОЕ SIGNALS И SLOTS?

Аналогия: РАЦИЯ

SIGNAL = Говоришь в рацию "Я нашёл клад!"

↓

SLOT = Кто-то слышит и реагирует "Иду к тебе!"

В Qt:

- **Signal** = событие ("что-то случилось!")
 - **Slot** = функция которая реагирует на событие
-

ШАГ 1: СОЗДАТЬ C++ КЛАСС С СИГНАЛАМИ

timeline.h:

cpp

```

#ifndef TIMELINE_H
#define TIMELINE_H

#include <QObject>

class Timeline : public QObject // ← ОБЯЗАТЕЛЬНО наследовать QObject!
{
    Q_OBJECT // ← ОБЯЗАТЕЛЬНЫЙ МАКРОС! Без него signals не работают!

    // ===== Q_PROPERTY (связь с QML) =====
    Q_PROPERTY(double currentTime
                READ currentTime
                WRITE setCurrentTime
                NOTIFY currentTimeChanged)

public:
    explicit Timeline(QObject *parent = nullptr);

    // ===== ГЕТТЕРЫ/СЕТТЕРЫ =====
    double currentTime() const { return m_currentTime; }
    void setCurrentTime(double time);

    // ===== Q_INVOKABLE (вызов из QML) =====
    Q_INVOKABLE bool addClip(const QString& filepath, int track, double time);

signals: // ← Секция СИГНАЛОВ (НЕ РЕАЛИЗУЮТСЯ!)
    void currentTimeChanged(); // Без параметров
    void clipAdded(int index); // С параметром
    void clipsChanged();

private:
    double m_currentTime;
};

#endif

```

ВАЖНО:

- **Q_OBJECT** - ОБЯЗАТЕЛЕН в начале класса!
- **signals:** - ТОЛЬКО ОБЪЯВЛЕНИЯ, БЕЗ тела функции!
- Сигналы НЕ РЕАЛИЗУЮТСЯ в .cpp файле!

ШАГ 2: РЕАЛИЗОВАТЬ КЛАСС

timeline.cpp:

```
cpp

#include "timeline.h"

Timeline::Timeline(QObject *parent)
: QObject(parent)
, m_currentTime(0.0)
{
}

void Timeline::setCurrentTime(double time) {
    if (m_currentTime == time) {
        return; // Не изменилось - не уведомляем
    }

    m_currentTime = time;

    // ===== ИСПУСКАЕМ СИГНАЛ! =====
    emit currentTimeChanged(); // ← emit = отправить сигнал!
    //   ↑           ↑
    // Ключевое   Имя сигнала
    // слово
}

bool Timeline::addClip(const QString& filepath, int track, double time) {
    // ... логика добавления ...

    m_clips.append(newClip);

    // ===== ИСПУСКАЕМ НЕСКОЛЬКО СИГНАЛОВ! =====
    emit clipsChanged(); // Общий сигнал
    emit clipAdded(m_clips.size() - 1); // С параметром!
    //           ↑
    // Передаём индекс нового клипа

    return true;
}
```

КОГДА ВЫЗЫВАТЬ emit?

- ☒ Изменили данные → emit сигнал
- ☒ После добавления/удаления → emit сигнал

- ☒ После любого изменения которое UI должен знать

🔗 ШАГ 3: ЗАРЕГИСТРИРОВАТЬ В QML

main.cpp:

```
cpp

#include <QGuiApplication>
#include <QQmlApplicationEngine>
#include <QQmlContext> // ← ОБЯЗАТЕЛЬНО!
#include "timeline.h"

int main(int argc, char *argv[]) {
    QGuiApplication app(argc, argv);

    // ===== СОЗДАТЬ C++ ОБЪЕКТ =====
    Timeline timeline; // ← Создали объект

    // ===== СОЗДАТЬ QML ENGINE =====
    QQmlApplicationEngine engine;

    // ===== ЗАРЕГИСТРИРОВАТЬ В QML =====
    engine.rootContext()->setContextProperty("cppTimeline", &timeline);
    //           ↑           ↑
    //           Имя в QML   Адрес объекта

    // ===== ЗАГРУЗИТЬ QML =====
    engine.load(QUrl(QStringLiteral("qrc:/main.qml")));

    return app.exec();
}
```

ЧТО ПРОИСХОДИТ:

1. Timeline timeline;
→ Создали C++ объект
2. setContextProperty("cppTimeline", &timeline)
→ Сделали его доступным в QML под именем "cppTimeline"
3. Теперь в QML можем:
cppTimeline.addClip(...)
cppTimeline.currentTime = 10

ПОЧЕМУ "cppTimeline"?

Это просто имя! Можешь назвать как хочешь:

```
cpp

setContextProperty("myTimeline", &timeline); // OK
setContextProperty("backend", &timeline);    // OK
setContextProperty("cppTimeline", &timeline); // OK (рекомендуется)
```

Я использую префикс `cpp` чтобы в QML было понятно: "Это C++ объект!"

ГДЕ ЭТО ПИСАТЬ?

ТОЛЬКО В main.cpp!

- ❌ НЕ в timeline.cpp
- ❌ НЕ в main.qml
- ✅ ТОЛЬКО в main.cpp

ОДИН РАЗ при старте программы!

🧩 ШАГ 4: ИСПОЛЬЗОВАТЬ В QML

Вариант 1: Прямой вызов

```
qml

// В любом QML файле:
Button {
    text: "Добавить клип"
    onClicked: {
        // ===== ВЫЗОВ C++ ФУНКЦИИ! =====
        cppTimeline.addClip("video.mp4", 1, 0.0)
        //   ↑       ↑
        // Имя из  Q_INVOKABLE
        // main.cpp функция
    }
}
```

Вариант 2: Connections (слушать сигналы)

```
qml
```

```
// ClipManager.qml
QObject {
    id: root

    // ===== СЛУШАЕМ СИГНАЛЫ ОТ C++! =====

    Connections {
        target: cppTimeline // ← От кого слушаем (имя из main.cpp)

        // ===== ОБРАБОТЧИКИ СИГНАЛОВ =====
        function onClipsChanged() {
            console.log(" 🐞 C++ сказал: клипы изменились!")
            root.updateUI()
        }

        function onClipAdded(index) {
            //      ↑      ↑
            //   on + Имя  Параметр
            //   сигнала   из C++
            console.log(" 🐞 C++ добавил клип", index)
        }
    }

    function updateUI() {
        // Обновить интерфейс
    }
}
```

ПРАВИЛО ИМЕНОВАНИЯ:

```
cpp
// C++:
signals:
    void clipAdded(int index);

// QML:
function onClipAdded(index) { // ← on + ClipAdded (CamelCase!)
    //      ↑
    //   Параметр из C++
}
```

Вариант 3: Q_PROPERTY (автобиндинг)

```
qml
```

```
// Автоматическое обновление!
Text {
    text: cppTimeline.currentTime // ← Автоматически обновляется!
    // ↑      ↑
    // C++ объект Q_PROPERTY
}

// Когда C++ вызывает:
emit currentTimeChanged();

// QML АВТОМАТИЧЕСКИ обновит Text!
```

ПОЛНЫЙ ЦИКЛ (от начала до конца)

1. Пользователь кликает в QML:

```
qml

Button {
    onClicked: {
        cppTimeline.addClip("video.mp4", 1, 0.0) // ← НАЧАЛО
    }
}
```

2. QML вызывает C++ функцию:

```
cpp

bool Timeline::addClip(const QString& filepath, int track, double time) {
    // Выполняем логику
    m_clips.append(newClip);

    // ИСПУСКАЕМ СИГНАЛ
    emit clipsChanged(); // ← ОТПРАВИЛИ В РАЦИЮ!

    return true;
}
```

3. Qt внутренне находит подписчиков:

Qt смотрит: "Кто подписан на signal clipsChanged()?"
Находит: Connections в QML
Вызывает: onClipsChanged()

4. QML получает сигнал:

```
qml

Connections {
    target: cppTimeline

    function onClipsChanged() { // ← ПОЛУЧИЛИ ИЗ РАЦИИ!
        console.log("Обновляю UI!")
        updateUI() // Обновляем интерфейс
    }
}
```

5. UI обновляется! ✓



СХЕМА ПОТОКА

```
USER ACTION (QML)
↓
cppTimeline.addClip() ← Вызов Q_INVOKABLE функции
↓
C++ Timeline::addClip() ← Выполняется код
↓
m_clips.append(clip) ← Изменяются данные
↓
emit clipsChanged() ← ОТПРАВКА СИГНАЛА!
↓
Qt внутренний механизм ← Qt находит подписчиков
↓
QML Connections::onClipsChanged() ← ПОЛУЧЕНИЕ СИГНАЛА!
↓
updateUI() ← Обновление интерфейса
↓
DONE! ✓
```



ПРИМЕРЫ ИСПОЛЬЗОВАНИЯ

Пример 1: Добавление клипа

C++:

```
cpp
```

```

bool Timeline::addClip(...) {
    m_clips.append(newClip);

    emit clipsChanged(); // "Что-то изменилось!"
    emit clipAdded(5);   // "Добавлен клип №5!"

    return true;
}

```

QML:

```

qml

Button {
    text: "Добавить"
    onClicked: cppTimeline.addClip("video.mp4", 1, 0)
}

Connections {
    target: cppTimeline

    function onClipsChanged() {
        console.log("Клипы изменились!")
    }

    function onClipAdded(index) {
        console.log("Добавлен клип", index)
    }
}

```

Пример 2: Изменение времени

C++:

```

cpp

void Timeline::setCurrentTime(double time) {
    m_currentTime = time;
    emit currentTimeChanged(); // NOTIFY!
}

```

QML:

```

qml

```

```
// Автоматическое обновление через Q_PROPERTY!
Text {
    text: "Time: " + cppTimeline.currentTime
}

Slider {
    onMoved: {
        cppTimeline.currentTime = value // Вызывает setCurrentTime()
    }
}
```

Пример 3: Рендеринг с прогрессом

C++:

```
cpp

bool Timeline::renderToFile(const QString& path) {
    for (int i = 0; i <= 100; i++) {
        // ... рендеринг ...

        emit renderProgress(i); // Отправляем прогресс!

        QThread::msleep(10);
    }

    emit renderFinished(true); // Завершено!
    return true;
}
```

QML:

```
qml
```

```

Dialog {
    id: exportDialog

    ProgressBar {
        id: progressBar
        value: 0
    }

    Button {
        text: "Экспорт"
        onClicked: {
            cppTimeline.renderToFile("output.mp4")
        }
    }
}

Connections {
    target: cppTimeline

    function onRenderProgress(percent) {
        progressBar.value = percent / 100
    }

    function onRenderFinished(success) {
        if (success) {
            console.log("✅ Готово!")
        }
        exportDialog.close()
    }
}

```

✖ ЧАСТЫЕ ОШИБКИ

Ошибка 1: Забыли Q_OBJECT

```

cpp

class Timeline : public QObject {
    // ✖ Забыли Q_OBJECT!

signals:
    void clipsChanged(); // НЕ БУДЕТ РАБОТАТЬ!
};

```

РЕШЕНИЕ: Всегда добавляй `Q_OBJECT` в начало класса!

Ошибка 2: Реализовали signal в .cpp

```
cpp

// timeline.cpp
void Timeline::clipsChanged() { // ❌ ОШИБКА!
    // Сигналы НЕ РЕАЛИЗУЮТСЯ!
}
```

РЕШЕНИЕ: Сигналы ТОЛЬКО объявляются в .h, НИКОГДА не реализуются!

Ошибка 3: Забыли emit

```
cpp

bool Timeline::addClip(...) {
    m_clips.append(newClip);

    // ❌ Забыли emit!
    // UI НЕ УЗНАЕТ об изменениях!

    return true;
}
```

РЕШЕНИЕ: Всегда `emit` после изменения данных!

Ошибка 4: Неправильное имя в QML

```
cpp

// C++:
signals:
    void clipAdded(int index);

// QML:
function onClipAdd(index) { // ❌ НЕПРАВИЛЬНО!
    // Должно быть onClipAdded (on + точное имя сигнала)
}
```

РЕШЕНИЕ: `on + ИмяСигнала` (CamelCase!)

✅ ЧЕКЛИСТ

Чтобы signals/slots работали:

- ☐ Класс наследует `QObject`
 - ☐ Добавлен `Q_OBJECT` макрос в начале класса
 - ☐ Сигналы объявлены в секции `signals:`
 - ☐ Сигналы НЕ реализованы в `.cpp`
 - ☐ Используется `emit` при отправке сигнала
 - ☐ Объект зарегистрирован в `main.cpp` через `setContextProperty`
 - ☐ В QML используется `Connections` с правильным `target`
 - ☐ Имена обработчиков: `on + ИмяСигнала`
 - ☐ `Q_INVOKABLE` перед функциями которые вызываются из QML
-

ИТОГО

Signals & Slots - это:

- Способ связи между объектами
- C++ → QML коммуникация
- Событийно-ориентированная архитектура
- Основа Qt Framework

Три кита:

1. **Q_OBJECT** - делает класс "Qt-совместимым"
2. **emit signal()** - отправка события
3. **Connections** - получение события в QML

ЭТО ПРОСТО РАЦИЯ! 📻